

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Joel Duke
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 51
CLASS: S3 CSE B	ROLE / FUNCTION: Mathematical Research & Problem Formulation

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.
3. Modular Inverses:
 - $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
 - $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
 - $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$
4. Compute x :
 - $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
 - $x = 140 + 63 + 30 = 233$

• $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

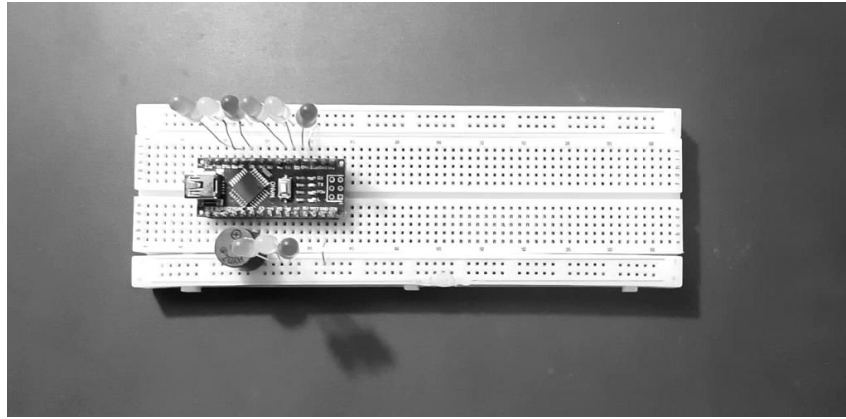
The unique green-wave solution within [0, 104] is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem> (Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
```

```

pinMode(RED_A, OUTPUT);
pinMode(YELLOW_A, OUTPUT);
pinMode(GREEN_A, OUTPUT);

pinMode(RED_B, OUTPUT);
pinMode(YELLOW_B, OUTPUT);
pinMode(GREEN_B, OUTPUT);

pinMode(RED_C, OUTPUT);
pinMode(YELLOW_C, OUTPUT);
pinMode(GREEN_C, OUTPUT);

pinMode(BUZZER, OUTPUT);

setAllRed();

tone(BUZZER, SYNC_FREQ);
syncActive = true;
syncEndTime = millis() + SYNC_TIME;

lastMillis = millis();
beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Joel Duke

Class: S3 CSE B

Roll No: 51

Role: Mathematical Research & Problem Formulation

Contribution Summary:

I researched the mathematical foundation of linear congruences and modular arithmetic for traffic signal timing cycles. I formulated the 3-intersection system equations for 3s, 5s, and 7s cycle lengths with non-zero remainder offsets, proved coprimality of moduli ($\gcd=1$), and computed modular inverses ($y_1=2$, $y_2=1$, $y_3=1$) using the Extended Euclidean Algorithm to solve for the unique green-wave timestamp ($t=23s$) within the 105s hyperperiod.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Joel Geo Manuel
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 52
CLASS: S3 CSE B	ROLE / FUNCTION: Presentation & Showcase Lead

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

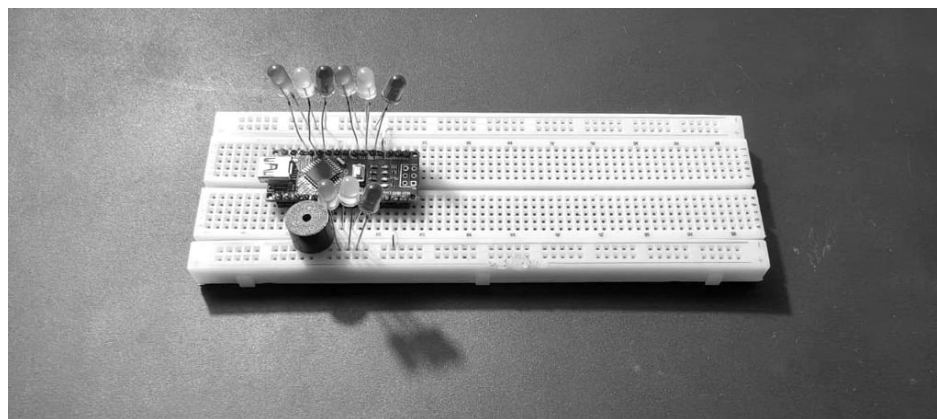
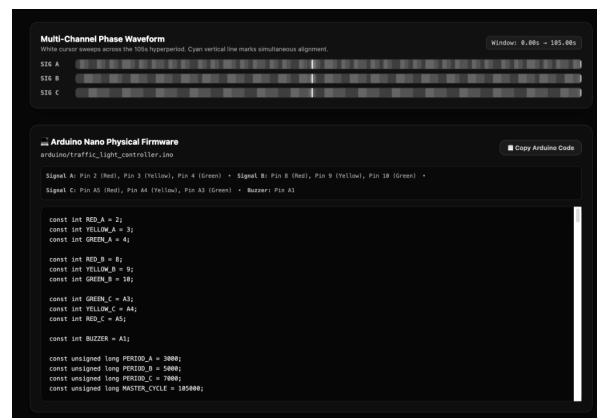
- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

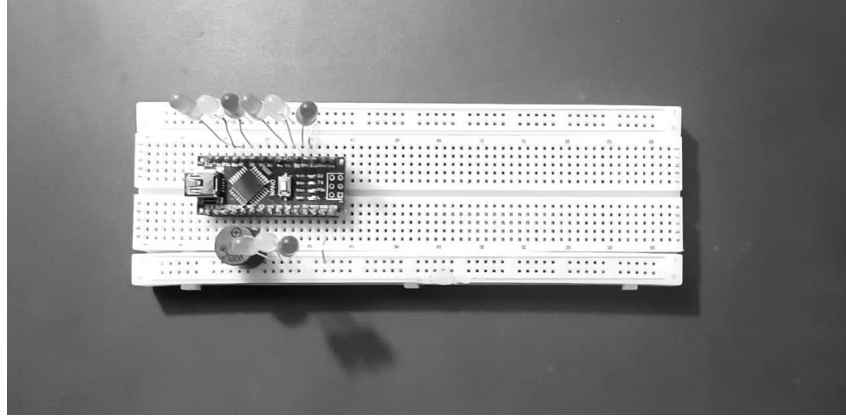
3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.
3. Modular Inverses:
 - $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
 - $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
 - $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$
4. Compute x :
 - $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
 - $x = 140 + 63 + 30 = 233$

The unique green-wave solution within $[0, 104]$ is Timestamp 23s.

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

[illegible]



Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
```

```

pinMode(RED_A, OUTPUT);
pinMode(YELLOW_A, OUTPUT);
pinMode(GREEN_A, OUTPUT);

pinMode(RED_B, OUTPUT);
pinMode(YELLOW_B, OUTPUT);
pinMode(GREEN_B, OUTPUT);

pinMode(RED_C, OUTPUT);
pinMode(YELLOW_C, OUTPUT);
pinMode(GREEN_C, OUTPUT);

pinMode(BUZZER, OUTPUT);

setAllRed();

tone(BUZZER, SYNC_FREQ);
syncActive = true;
syncEndTime = millis() + SYNC_TIME;

lastMillis = millis();
beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Joel Geo Manuel

Class: S3 CSE B

Roll No: 52

Role: Presentation & Showcase Lead

Contribution Summary:

I led the presentation design and showcase workflow for explaining the traffic signal synchronization concept to live audiences. I authored the 2-speaker presentation dialogue between driver Alex and tech lead Sam, structured the live demo pacing around 3s/5s/7s signal cycles, prepared the slide deck visual layout, and coordinated acoustic chime feedback during live hardware and digital twin demonstrations.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Johaan Sam
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 53
CLASS: S3 CSE B	ROLE / FUNCTION: Frontend & State Engine Developer

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.
3. Modular Inverses:
 - $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
 - $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
 - $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$
4. Compute x :
 - $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
 - $x = 140 + 63 + 30 = 233$

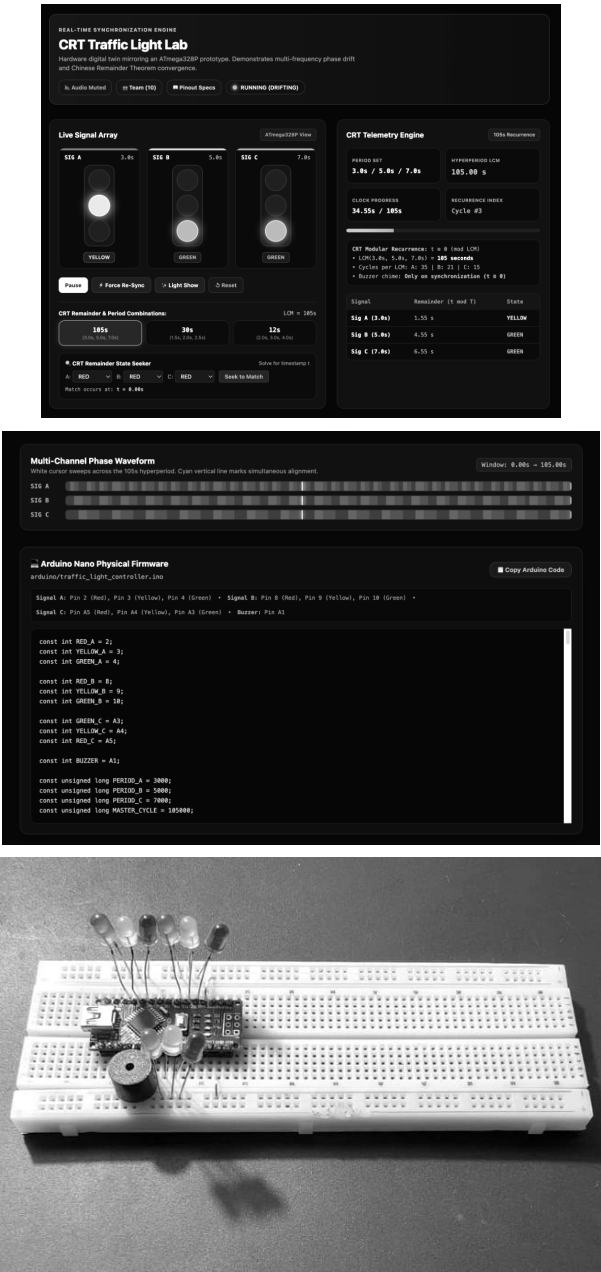
• $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

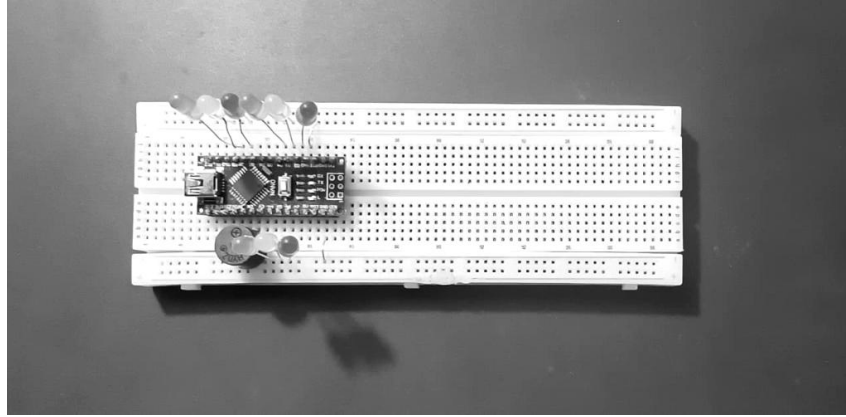
The unique green-wave solution within [0, 104] is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem> (Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
```

```

pinMode(RED_A, OUTPUT);
pinMode(YELLOW_A, OUTPUT);
pinMode(GREEN_A, OUTPUT);

pinMode(RED_B, OUTPUT);
pinMode(YELLOW_B, OUTPUT);
pinMode(GREEN_B, OUTPUT);

pinMode(RED_C, OUTPUT);
pinMode(YELLOW_C, OUTPUT);
pinMode(GREEN_C, OUTPUT);

pinMode(BUZZER, OUTPUT);

setAllRed();

tone(BUZZER, SYNC_FREQ);
syncActive = true;
syncEndTime = millis() + SYNC_TIME;

lastMillis = millis();
beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Johaan Sam

Class: S3 CSE B

Roll No: 53

Role: Frontend & State Engine Developer

Contribution Summary:

I engineered the interactive web application frontend and state engine using React 19 and TypeScript. I developed custom 60 FPS animation loops and telemetry sweep cursors, implemented the dynamic CRT remainder state solver, wired real-time state synchronization across traffic monitors and phase waveforms, and tuned Framer Motion visual transitions for seamless user interaction.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Johan Geo
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 54
CLASS: S3 CSE B	ROLE / FUNCTION: Web Design & UI Architect

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.
3. Modular Inverses:
 - $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
 - $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
 - $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$
4. Compute x :
 - $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
 - $x = 140 + 63 + 30 = 233$

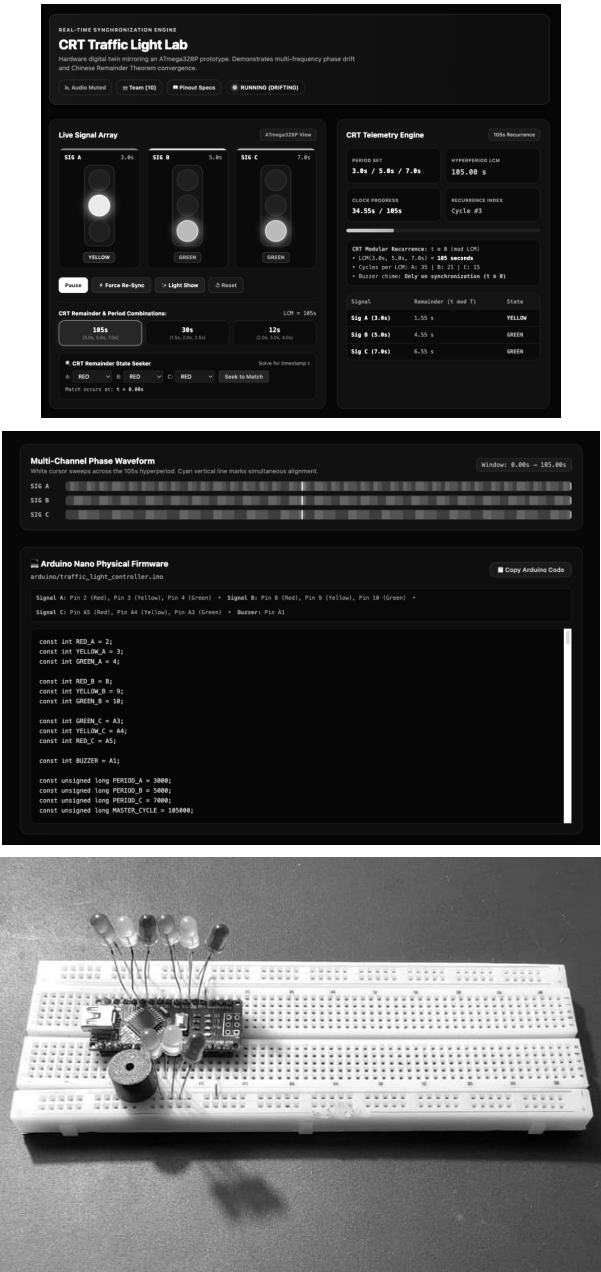
• $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

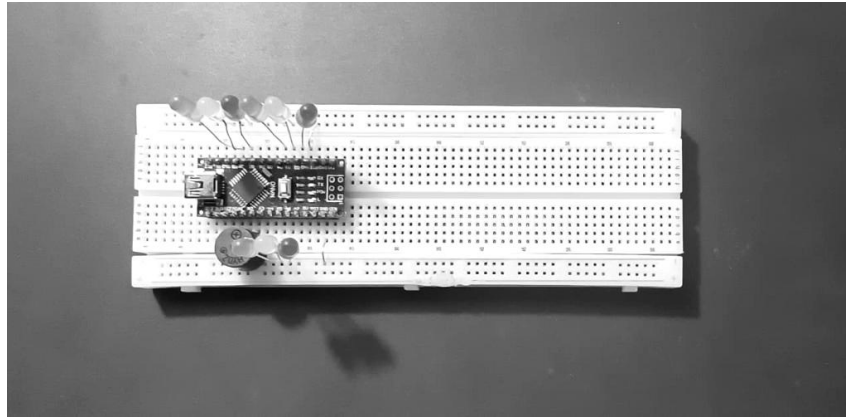
The unique green-wave solution within [0, 104] is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem> (Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
```

```

pinMode(RED_A, OUTPUT);
pinMode(YELLOW_A, OUTPUT);
pinMode(GREEN_A, OUTPUT);

pinMode(RED_B, OUTPUT);
pinMode(YELLOW_B, OUTPUT);
pinMode(GREEN_B, OUTPUT);

pinMode(RED_C, OUTPUT);
pinMode(YELLOW_C, OUTPUT);
pinMode(GREEN_C, OUTPUT);

pinMode(BUZZER, OUTPUT);

setAllRed();

tone(BUZZER, SYNC_FREQ);
syncActive = true;
syncEndTime = millis() + SYNC_TIME;

lastMillis = millis();
beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Johan Geo

Class: S3 CSE B

Roll No: 54

Role: Web Design & UI Architect

Contribution Summary:

I designed the complete visual architecture, color palette, dark laboratory theme, and responsive layout system using Tailwind CSS v4. I created custom SVG circular monitor displays for 3-intersection traffic signals, designed interactive dial controls and status panels, and established high-contrast UI component states for optimal visual clarity across desktop and mobile displays.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Jose Alex
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 55
CLASS: S3 CSE B	ROLE / FUNCTION: Mathematical Proofs & Solution Verification

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.
3. Modular Inverses:
 - $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
 - $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
 - $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$
4. Compute x :
 - $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
 - $x = 140 + 63 + 30 = 233$

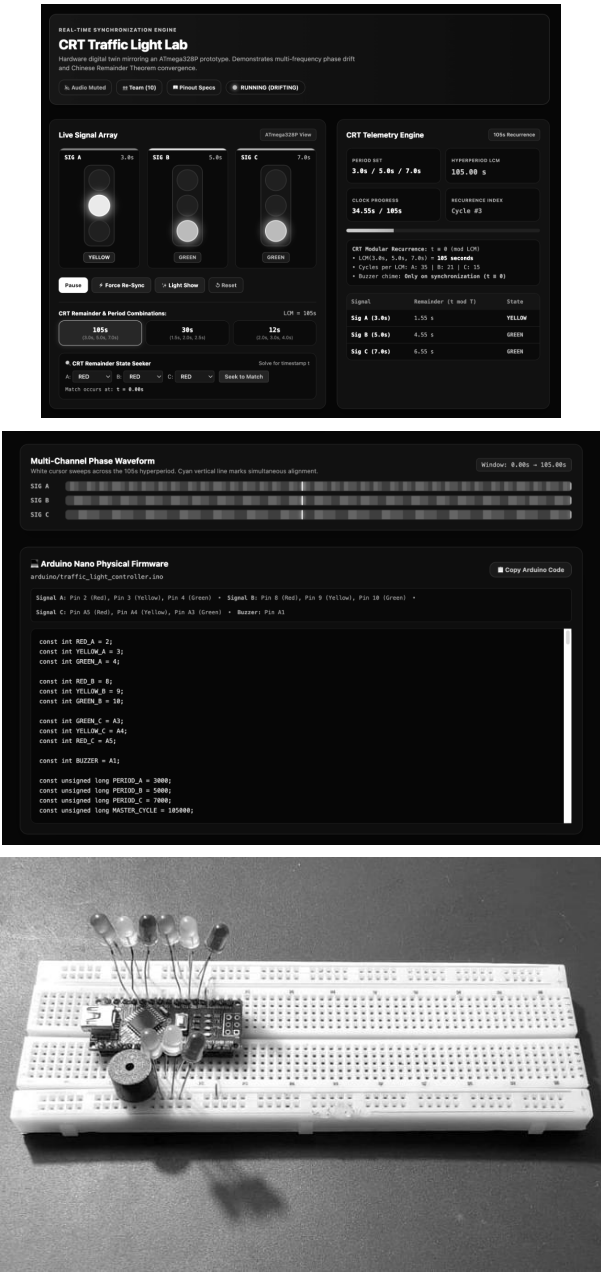
• $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

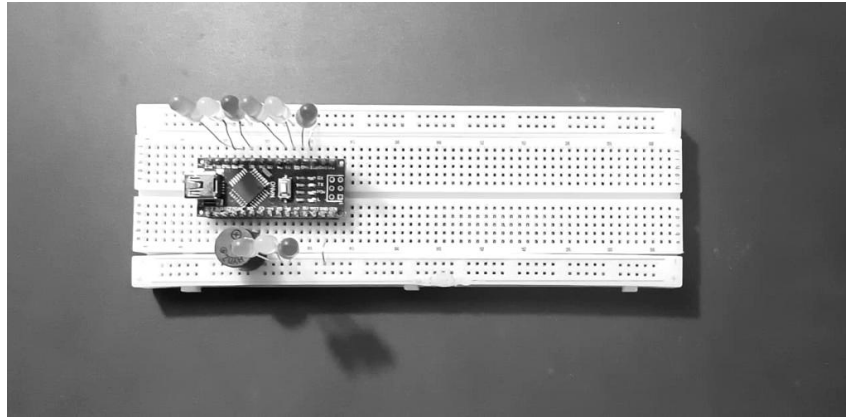
The unique green-wave solution within [0, 104] is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem> (Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
```

```

pinMode(RED_A, OUTPUT);
pinMode(YELLOW_A, OUTPUT);
pinMode(GREEN_A, OUTPUT);

pinMode(RED_B, OUTPUT);
pinMode(YELLOW_B, OUTPUT);
pinMode(GREEN_B, OUTPUT);

pinMode(RED_C, OUTPUT);
pinMode(YELLOW_C, OUTPUT);
pinMode(GREEN_C, OUTPUT);

pinMode(BUZZER, OUTPUT);

setAllRed();

tone(BUZZER, SYNC_FREQ);
syncActive = true;
syncEndTime = millis() + SYNC_TIME;

lastMillis = millis();
beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Jose Alex

Class: S3 CSE B

Roll No: 55

Role: Mathematical Proofs & Solution Verification

Contribution Summary:

I verified the mathematical proofs, solution existence, and uniqueness bounds for system congruences modulo $M=105$. I conducted formal step-by-step calculations for Extended Euclidean Algorithm modular inverses (y_1, y_2, y_3) , validated remainder offset equations against theoretical bounds, and analyzed boundary edge cases to ensure mathematical rigor in all solution derivations.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Joseph Alex
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 56
CLASS: S3 CSE B	ROLE / FUNCTION: Hardware Sourcing & Build Testing

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.
3. Modular Inverses:
 - $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
 - $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
 - $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$
4. Compute x :
 - $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
 - $x = 140 + 63 + 30 = 233$

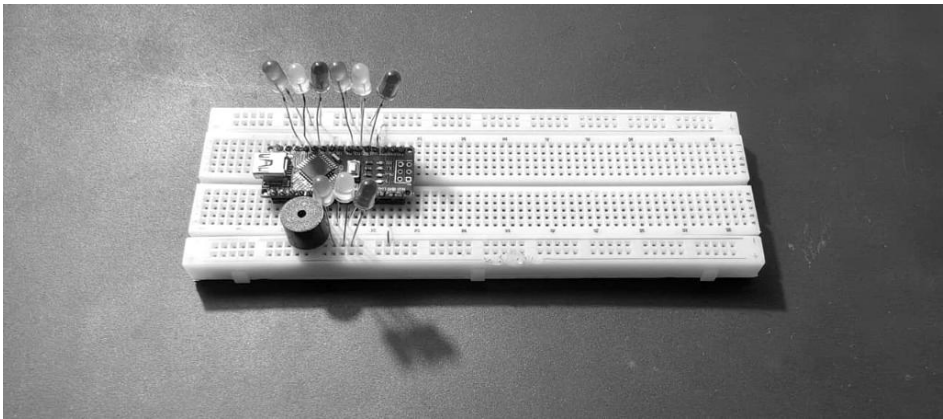
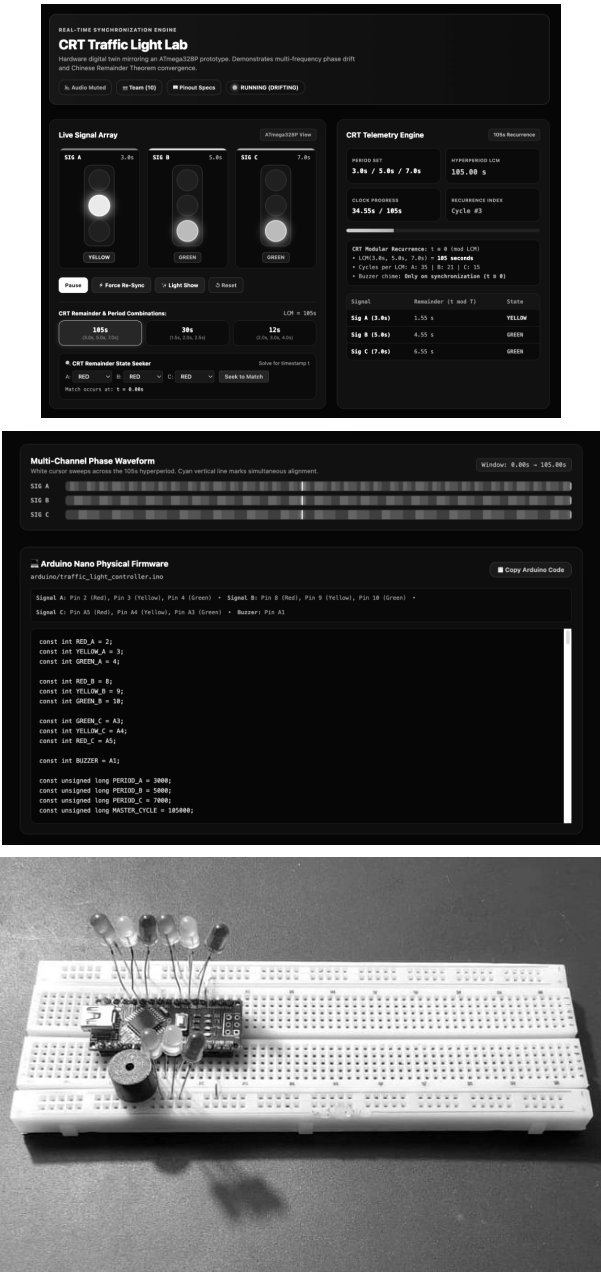
• $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

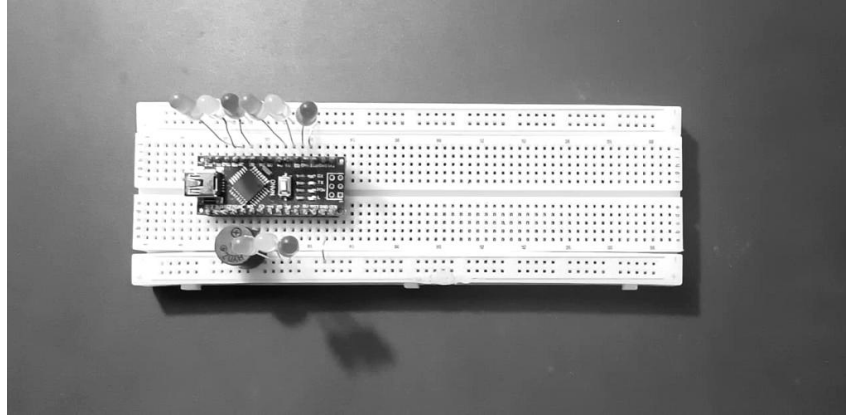
The unique green-wave solution within [0, 104] is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem> (Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
```

```

pinMode(RED_A, OUTPUT);
pinMode(YELLOW_A, OUTPUT);
pinMode(GREEN_A, OUTPUT);

pinMode(RED_B, OUTPUT);
pinMode(YELLOW_B, OUTPUT);
pinMode(GREEN_B, OUTPUT);

pinMode(RED_C, OUTPUT);
pinMode(YELLOW_C, OUTPUT);
pinMode(GREEN_C, OUTPUT);

pinMode(BUZZER, OUTPUT);

setAllRed();

tone(BUZZER, SYNC_FREQ);
syncActive = true;
syncEndTime = millis() + SYNC_TIME;

lastMillis = millis();
beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Joseph Alex

Class: S3 CSE B

Roll No: 56

Role: Hardware Sourcing & Build Testing

Contribution Summary:

I managed the selection and procurement of physical electronic components, acquired Arduino Nano microcontrollers, breadboards, 220 ohm resistors, 9 LED traffic clusters, and piezo buzzers. I executed bench testing, verified digital and analog GPIO pin voltage outputs, conducted breadboard wiring continuity checks, and validated hardware reliability during continuous operation.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Joseph J
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 57
CLASS: S3 CSE B	ROLE / FUNCTION: Component Procurement & Application Ideas

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.
3. Modular Inverses:
 - $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
 - $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
 - $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$
4. Compute x :
 - $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
 - $x = 140 + 63 + 30 = 233$

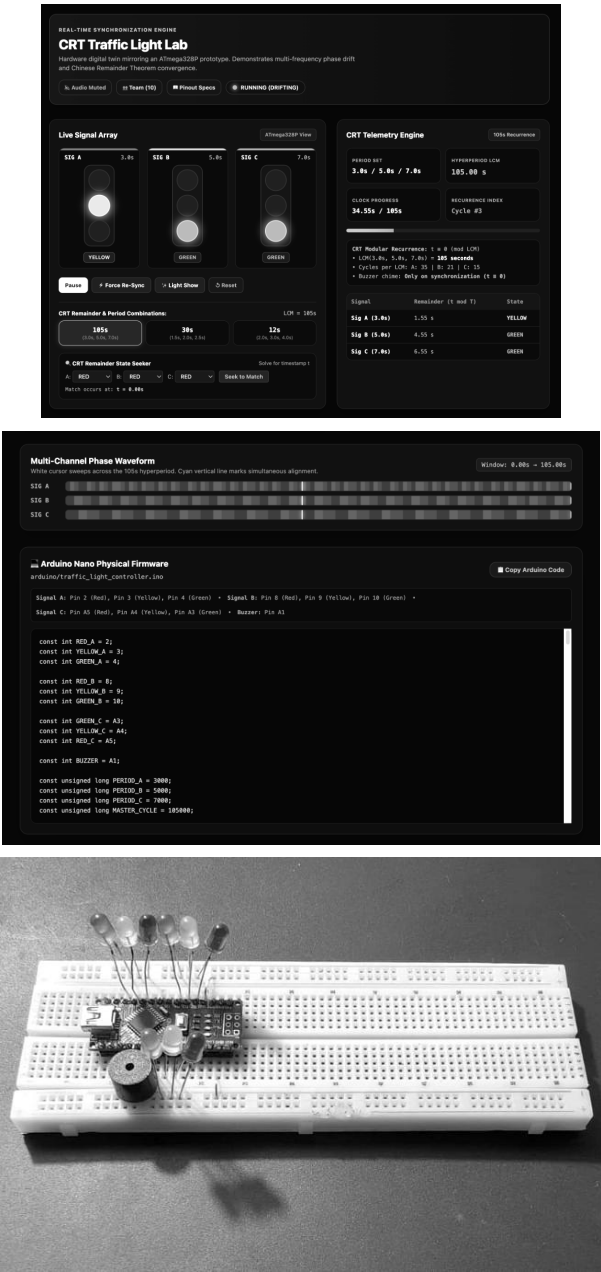
• $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

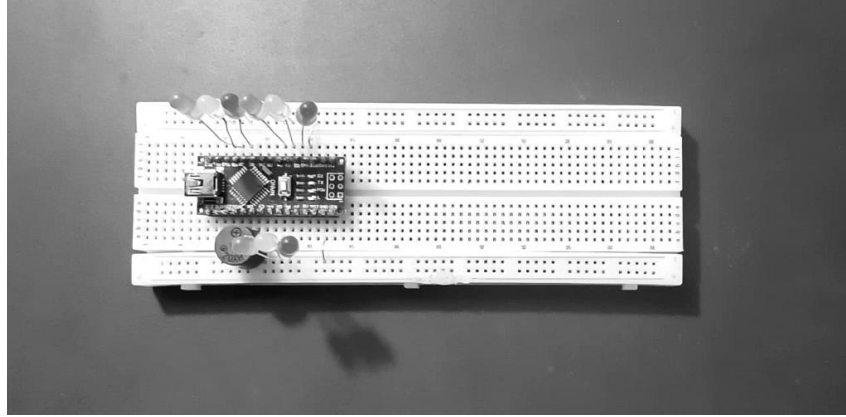
The unique green-wave solution within [0, 104] is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem> (Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
```

```

pinMode(RED_A, OUTPUT);
pinMode(YELLOW_A, OUTPUT);
pinMode(GREEN_A, OUTPUT);

pinMode(RED_B, OUTPUT);
pinMode(YELLOW_B, OUTPUT);
pinMode(GREEN_B, OUTPUT);

pinMode(RED_C, OUTPUT);
pinMode(YELLOW_C, OUTPUT);
pinMode(GREEN_C, OUTPUT);

pinMode(BUZZER, OUTPUT);

setAllRed();

tone(BUZZER, SYNC_FREQ);
syncActive = true;
syncEndTime = millis() + SYNC_TIME;

lastMillis = millis();
beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Joseph J

Class: S3 CSE B

Roll No: 57

Role: Component Procurement & Application Ideas

Contribution Summary:

I handled hardware component selection, part procurement, and application domain research. I mapped Chinese Remainder Theorem modular congruences to real-world engineering problems, detailing how CRT timing algorithms are applied in urban traffic signal synchronization, fiber-optic telecommunications channel multiplexing, and parallel computing architectures.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Justin Joe Mathew
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 58
CLASS: S3 CSE B	ROLE / FUNCTION: Team Lead & Working Model Engineer

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.
3. Modular Inverses:
 - $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
 - $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
 - $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$
4. Compute x :
 - $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
 - $x = 140 + 63 + 30 = 233$

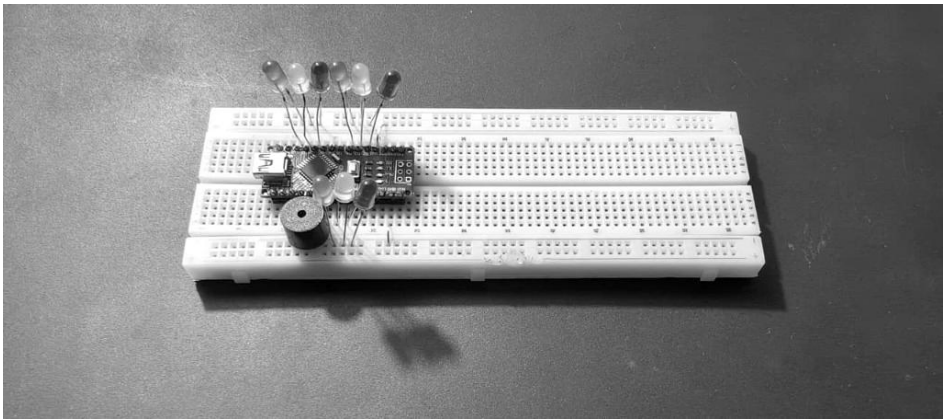
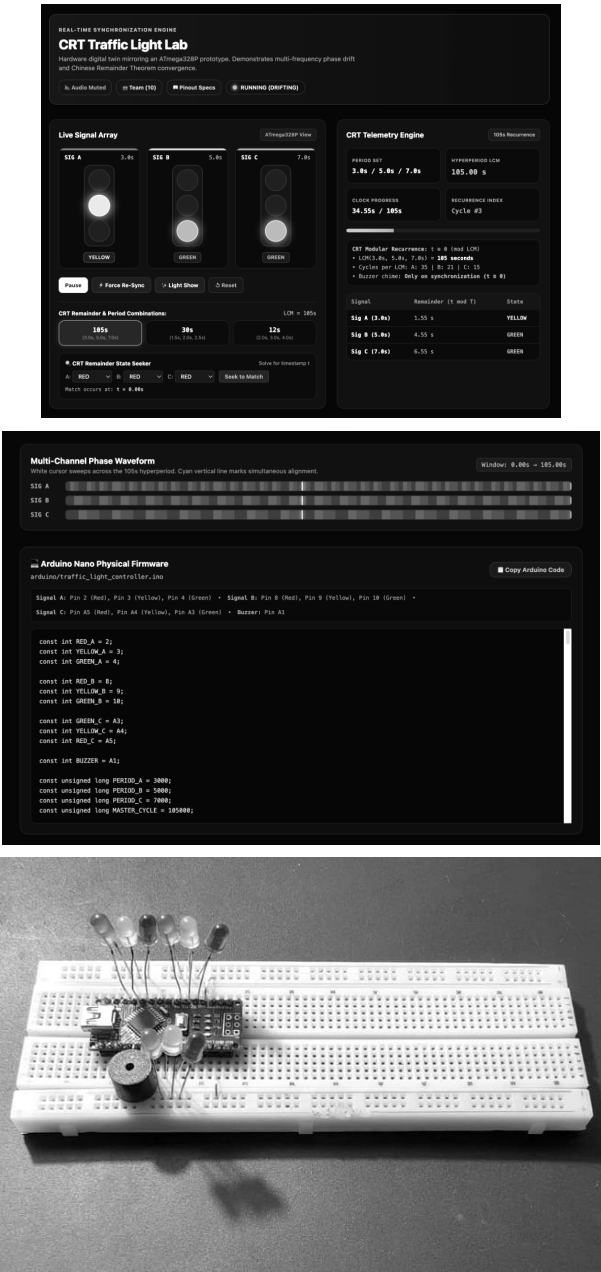
• $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

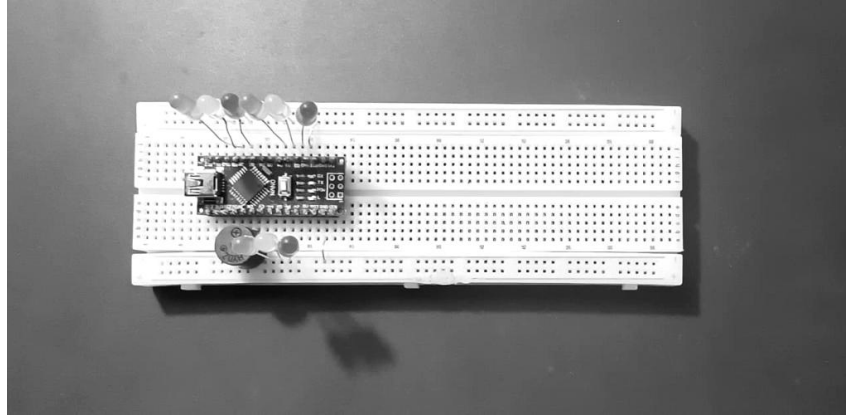
The unique green-wave solution within [0, 104] is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem> (Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
```

```

pinMode(RED_A, OUTPUT);
pinMode(YELLOW_A, OUTPUT);
pinMode(GREEN_A, OUTPUT);

pinMode(RED_B, OUTPUT);
pinMode(YELLOW_B, OUTPUT);
pinMode(GREEN_B, OUTPUT);

pinMode(RED_C, OUTPUT);
pinMode(YELLOW_C, OUTPUT);
pinMode(GREEN_C, OUTPUT);

pinMode(BUZZER, OUTPUT);

setAllRed();

tone(BUZZER, SYNC_FREQ);
syncActive = true;
syncEndTime = millis() + SYNC_TIME;

lastMillis = millis();
beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Justin Joe Mathew

Class: S3 CSE B

Roll No: 58

Role: Team Lead & Working Model Engineer

Contribution Summary:

I led overall project execution, system architecture, and physical model construction as Team Lead. I engineered the 3-intersection breadboard circuit, wrote the C++ firmware (arduino/traffic_light_controller.ino) for Arduino Nano microcontroller timing and acoustic chime feedback, and established full integration between the physical hardware prototype and the digital twin web application.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Jyothika Prakash
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 59
CLASS: S3 CSE B	ROLE / FUNCTION: Pedagogical Systems & Educational Design

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.
3. Modular Inverses:
 - $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
 - $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
 - $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$
4. Compute x :
 - $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
 - $x = 140 + 63 + 30 = 233$

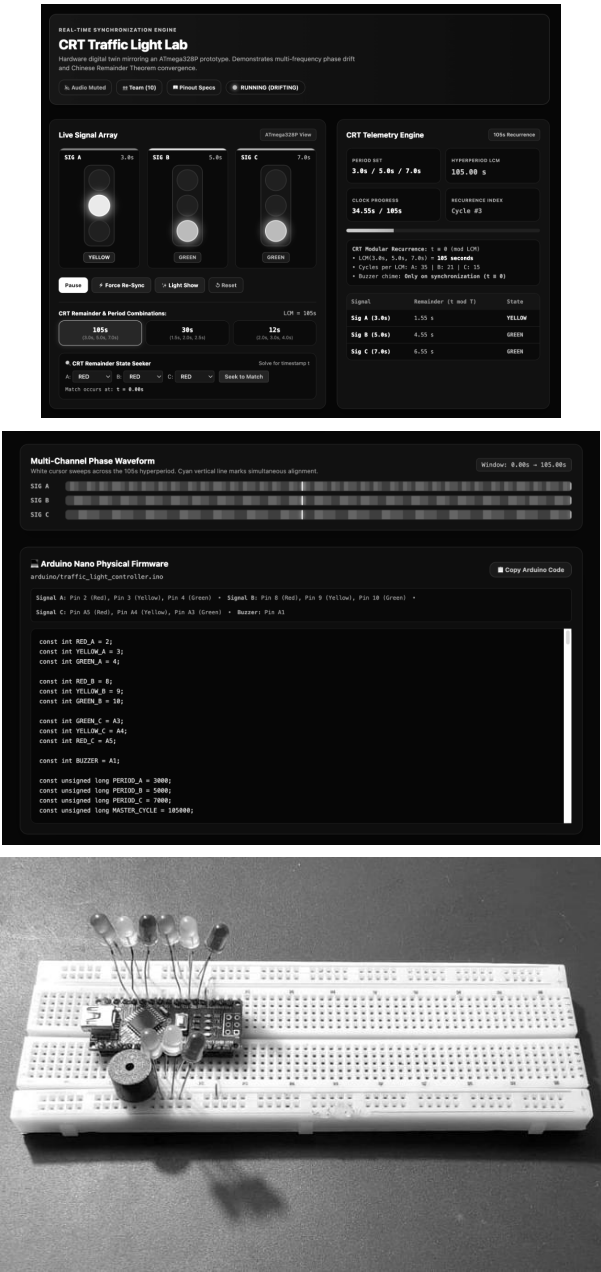
• $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

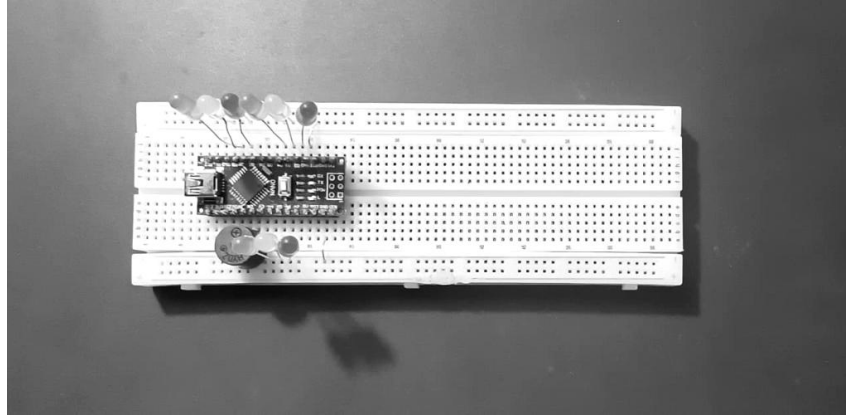
The unique green-wave solution within [0, 104] is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem> (Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
```

```

pinMode(RED_A, OUTPUT);
pinMode(YELLOW_A, OUTPUT);
pinMode(GREEN_A, OUTPUT);

pinMode(RED_B, OUTPUT);
pinMode(YELLOW_B, OUTPUT);
pinMode(GREEN_B, OUTPUT);

pinMode(RED_C, OUTPUT);
pinMode(YELLOW_C, OUTPUT);
pinMode(GREEN_C, OUTPUT);

pinMode(BUZZER, OUTPUT);

setAllRed();

tone(BUZZER, SYNC_FREQ);
syncActive = true;
syncEndTime = millis() + SYNC_TIME;

lastMillis = millis();
beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Jyothika Prakash

Class: S3 CSE B

Roll No: 59

Role: Pedagogical Systems & Educational Design

Contribution Summary:

I developed the pedagogical system, progressive 3-tier educational hint engine, and explanatory learning modules. I authored step-by-step documentation translating abstract number theory and modulo congruences $x \equiv a_i \pmod{m_i}$ into intuitive traffic light timing concepts, ensuring accessible comprehension for students and non-technical viewers.

MATHEMATICS PROJECT ASSIGNMENT REPORT

PROJECT TITLE: Traffic Signal Synchronization System Using Chinese Remainder Theorem	STUDENT NAME: Jyothis Liju
COURSE: Number Theory & Discrete Mathematics Application	ROLL NO: 60
CLASS: S3 CSE B	ROLE / FUNCTION: Working Model Co-Developer

1. ABSTRACT

The Chinese Remainder Theorem (CRT) is a fundamental result in number theory that determines a unique integer solution x for a system of linear congruences with pairwise coprime moduli. While often taught abstractly, CRT has direct applications in urban traffic signal synchronization, digital telecommunications, and parallel computer architectures.

This project demonstrates CRT using both a physical working Arduino Nano traffic signal model and an interactive digital twin web application. By modeling traffic signals as modular timing cycles, the project illustrates how independent light timers (3s, 5s, 7s) align to create a continuous 'Green Wave' for vehicles at Minute 23 (105s master recurrence).

2. INTRODUCTION & OBJECTIVES

When traffic signals operate on independent timer cycles along a main avenue, drivers frequently encounter red lights. Traffic engineers solve this by calculating when all signals will simultaneously show green.

Project Objectives:

1. To understand the mathematical principles of the Chinese Remainder Theorem (CRT).
2. To differentiate between Least Common Multiple (LCM) and CRT when remainder offsets are present.
3. To model traffic signal timing cycles (3s, 5s, 7s) as modular congruences.
4. To build a physical working model powered by an Arduino Nano microcontroller.
5. To provide clear visual, audio, and waveform telemetry feedback upon signal synchronization.

3. MATHEMATICAL FORMULATION (CRT vs. LCM)

3.1 The Traffic Signal Equations

Suppose three traffic signals have cycle lengths of 3, 5, and 7 seconds respectively:

- Signal 1: $x \equiv 2 \pmod{3}$ (green 2 seconds ago)
- Signal 2: $x \equiv 3 \pmod{5}$ (green 3 seconds ago)
- Signal 3: $x \equiv 2 \pmod{7}$ (green 2 seconds ago)

3.2 Why Simple LCM Cannot Solve This Problem

- LCM: Used only when all signals start at Minute 0 with zero remainder offset ($x \equiv 0$).
- CRT: Required when remainder offsets (2, 3, 2) are distinct and non-zero.

3.3 CRT Solution Procedure

1. Total Product: $M = 3 \times 5 \times 7 = 105$.
2. Partial Products: $M1 = 35$, $M2 = 21$, $M3 = 15$.
3. Modular Inverses:
 - $35 * y1 \equiv 1 \pmod{3} \Rightarrow 2 * y1 \equiv 1 \pmod{3} \Rightarrow y1 = 2$
 - $21 * y2 \equiv 1 \pmod{5} \Rightarrow 1 * y2 \equiv 1 \pmod{5} \Rightarrow y2 = 1$
 - $15 * y3 \equiv 1 \pmod{7} \Rightarrow 1 * y3 \equiv 1 \pmod{7} \Rightarrow y3 = 1$
4. Compute x :
 - $x = (2 \times 35 \times 2) + (3 \times 21 \times 1) + (2 \times 15 \times 1)$
 - $x = 140 + 63 + 30 = 233$

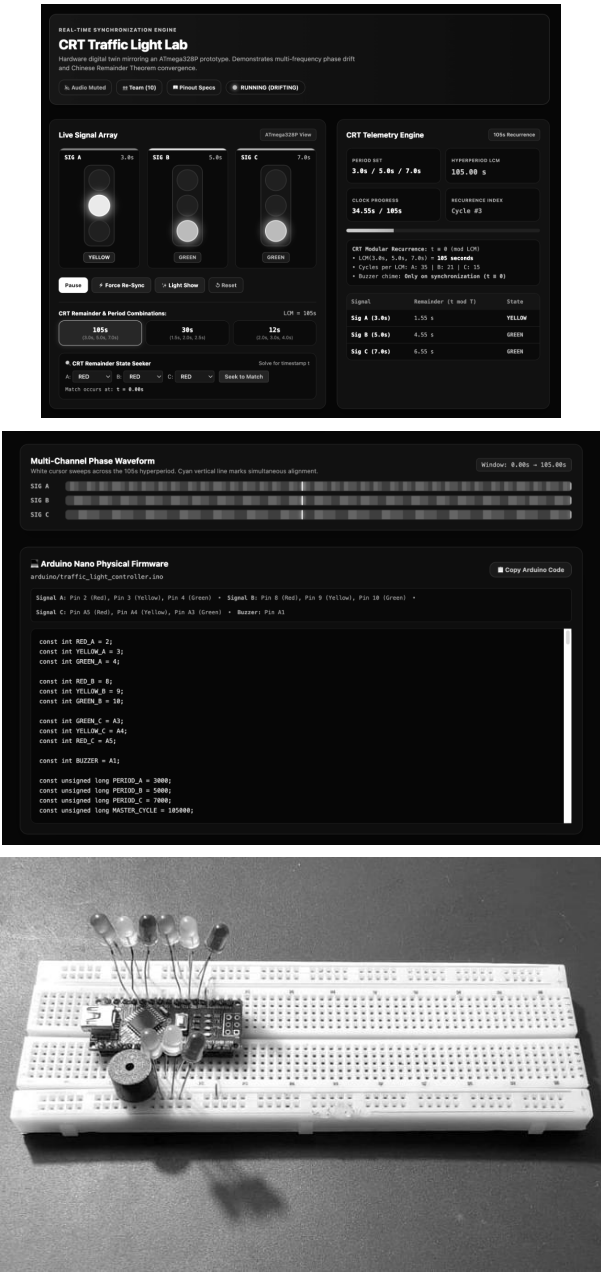
• $x \equiv 233 \pmod{105} \equiv 23 \pmod{105}$

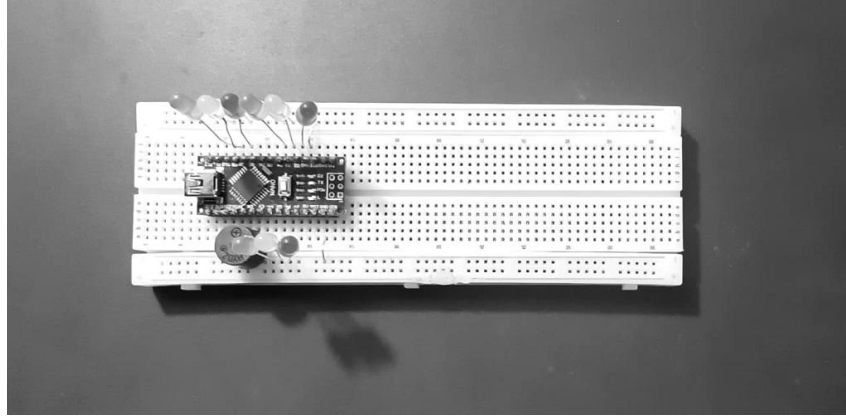
The unique green-wave solution within [0, 104] is Timestamp 23s.

4. PHYSICAL HARDWARE & DIGITAL TWIN IMPLEMENTATION

The hardware prototype is driven by an Arduino Nano microcontroller controlling a 3-intersection LED traffic array and piezo buzzer. The physical build is paired with an interactive digital twin web application accessible at:

■ Interactive Web App & Project Repository: <https://github.com/jvstin47/CRT--Chinese-Reminder-Theorem> (Local Dev Server: <http://localhost:5173/>)





Interactive Hardware & Digital Twin Controls:

1. Live Signal Array (SIG A 3.0s, SIG B 5.0s, SIG C 7.0s): Monitors real-time LED states across intersections.
 2. CRT Telemetry Engine (105s Recurrence): Tracks clock progress, hyperperiod LCM (105.00s), and recurrence index.
 3. Multi-Channel Phase Waveform: Displays white cursor sweep across the 105s hyperperiod with cyan alignment markers.
 4. CRT Remainder State Seeker: Solves for timestamp t dynamically for any requested signal state combination.
-

5. HARDWARE SOURCE CODE (ARDUINO NANO FIRMWARE - C++)

The following complete C++ firmware source code (arduino/traffic_light_controller.ino) is executed directly on the Arduino Nano microcontroller to drive the physical traffic light hardware:

```
// Arduino Nano Physical Traffic Light Controller
// File: arduino/traffic_light_controller.ino

const int RED_A = 2;
const int YELLOW_A = 3;
const int GREEN_A = 4;

const int RED_B = 8;
const int YELLOW_B = 9;
const int GREEN_B = 10;

const int GREEN_C = A3;
const int YELLOW_C = A4;
const int RED_C = A5;

const int BUZZER = A1;

// Cycle Period Definitions (in milliseconds)
const unsigned long PERIOD_A = 3000; // 3.0s Signal Cycle
const unsigned long PERIOD_B = 5000; // 5.0s Signal Cycle
const unsigned long PERIOD_C = 7000; // 7.0s Signal Cycle
const unsigned long MASTER_CYCLE = 105000; // 105.0s Hyperperiod LCM

const int RED_FREQ = 400;
const int YELLOW_FREQ = 600;
const int GREEN_FREQ = 800;
const int SYNC_FREQ = 1000;

const unsigned long BEEP_ON_TIME = 150;
const unsigned long BEEP_OFF_TIME = 350;
const unsigned long SYNC_TIME = 600;

unsigned long lastMillis = 0;
unsigned long lastSyncCycle = 0;
unsigned long beepTimer = 0;
unsigned long syncEndTime = 0;

bool beepState = false;
bool syncActive = false;
int currentBuzzerState = -1;

void setup() {
```

```

pinMode(RED_A, OUTPUT);
pinMode(YELLOW_A, OUTPUT);
pinMode(GREEN_A, OUTPUT);

pinMode(RED_B, OUTPUT);
pinMode(YELLOW_B, OUTPUT);
pinMode(GREEN_B, OUTPUT);

pinMode(RED_C, OUTPUT);
pinMode(YELLOW_C, OUTPUT);
pinMode(GREEN_C, OUTPUT);

pinMode(BUZZER, OUTPUT);

setAllRed();

tone(BUZZER, SYNC_FREQ);
syncActive = true;
syncEndTime = millis() + SYNC_TIME;

lastMillis = millis();
beepTimer = millis();
}

void loop() {
    unsigned long currentMillis = millis();

    // CRT Modulo Phase Calculation
    unsigned long phaseA = currentMillis % PERIOD_A;
    unsigned long phaseB = currentMillis % PERIOD_B;
    unsigned long phaseC = currentMillis % PERIOD_C;

    int stateA = updateLightA(phaseA);
    int stateB = updateLightB(phaseB);
    int stateC = updateLightC(phaseC);

    // Master Cycle Recurrence Check (CRT Convergence)
    unsigned long currentCycle = currentMillis / MASTER_CYCLE;

    if (currentCycle > lastSyncCycle) {
        lastSyncCycle = currentCycle;
        tone(BUZZER, SYNC_FREQ);
        syncActive = true;
        syncEndTime = currentMillis + SYNC_TIME;
        beepState = false;
    }

    if (syncActive) {
        if (currentMillis >= syncEndTime) {
            noTone(BUZZER);
            syncActive = false;
            beepTimer = currentMillis;
            beepState = false;
            currentBuzzerState = -1;
        }
        return;
    }
}

```

6. CONCLUSION

The Traffic Signal Synchronization System provides an intuitive, practical bridge between abstract number theory and urban infrastructure engineering. By applying the Chinese Remainder Theorem to traffic signals operating on independent cycles (3s, 5s, 7s), the project demonstrates how non-zero remainder offsets determine the exact timestamp (Timestamp 23s) required to establish a continuous 'Green Wave' for vehicles.

Both the physical Arduino Nano hardware prototype and the digital twin web application successfully visualize modular congruences, offering an accessible learning tool that highlights the real-world power of modular arithmetic in signal timing, telecommunications, and digital computing.

7. INDIVIDUAL CONTRIBUTION

Name: Jyothis Liju

Class: S3 CSE B

Roll No: 60

Role: Working Model Co-Developer

Contribution Summary:

I co-developed the physical working model and assisted in breadboard circuit assembly and microcontroller pin configuration. I wired digital output pins (Pins 2–10) and analog pins (A1, A3–A5), conducted real-time signal timing tests against C++ firmware modulo logic, and verified acoustic piezo buzzer chime synchronization at the 105s master hyperperiod.